Journal of Process Control 116 (2022) 80–92

Contents lists available at ScienceDirect

Journal of Process Control

journal homepage: www.elsevier.com/locate/jprocont

Differentiable predictive control: Deep learning alternative to explicit model predictive control for unknown nonlinear systems

Ján Drgoňa [a,*], Karol Kiš [b], Aaron Tuor [a], Draguna Vrabie [a], Martin Klaučo [b]

[a] Pacific Northwest National Laboratory, Richland, WA, USA
[b] Slovak University of Technology in Bratislava, Bratislava, Slovakia

ARTICLE INFO

Article history:
Received 29 June 2021
Received in revised form 21 April 2022
Accepted 1 June 2022
Available online 21 June 2022

Keywords:
Differentiable predictive control
Model predictive control
Neural state space models
Data-driven differentiable optimization
Deep learning

ABSTRACT

We present differentiable predictive control (DPC) as a deep learning-based alternative to the explicit model predictive control (MPC) for unknown nonlinear systems. In the DPC framework, a neural state-space model is learned from time-series measurements of the system dynamics. The neural control policy is then optimized via stochastic gradient descent approach by differentiating the MPC loss function through the closed-loop system dynamics model. The proposed DPC method learns model-based control policies with state and input constraints, while supporting time-varying references and constraints. In embedded implementation using a Raspberry-Pi platform, we experimentally demonstrate that it is possible to train constrained control policies purely based on the measurements of the unknown nonlinear system. We compare the control performance of the DPC method against explicit MPC and report efficiency gains in online computational demands, memory requirements, policy complexity, and construction time. In particular, we show that our method scales linearly compared to exponential scalability of the explicit MPC solved via multiparametric programming.

© 2022 Battelle Memorial Institute and The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. Introduction

Incorporation of machine learning methods in control applications is becoming one of the leading research avenues in the field of control theory. The design of many novel control methods based on machine learning (ML) approaches is heavily inspired by the benefits of model predictive control (MPC), such as constraints handling and robustness. The substitution of MPC with ML-based controllers was studied by several well-established researchers in the control domain [1–5]. Furthermore, the application of neural networks as substitutes of MPC behavior has been considered in practical applications as well [6–9]. All aforementioned works fall into the category of so-called approximate MPC based on imitation learning of original MPC. As such, these works have one significant disadvantage, which is the reliance on data sets collected from closed-loop experiments, i.e., the ML-based controllers are trained based on experiments involving fully implemented MPC.

Addressing these challenges, we present a novel control method called differentiable predictive control (DPC) for learning both nonlinear system dynamics and constrained control policies without supervision from the expert controller in a sample efficient way. The only requirement is a dataset of sufficiently excited time-series of the system dynamics. This in contrast with the recent implementation in [1], where a trained ML-based controller runs alongside an online MPC strategy, making the algorithm computationally expensive in many applications running in low computational resource settings. The presented DPC method is based on the neural parametrization of the closed-loop dynamical system with two building blocks: (i) neural system model, and (ii) a deep learning formulation of model predictive control policy. In particular, we combine system identification based on structured neural state-space model and policy optimization with embedded inequality constraints via backpropagation of the control loss through the closed-loop system dynamics model. The conceptual methodology of DPC is illustrated in Fig. 1.

The presented DPC method supports nonlinear systems, with both input and state constraints, arbitrary reference signals and represents a methodological extension of prior work on DPC for linear systems [10]. A method similar in spirit has been recently proposed by [11], where the authors use Log-Sum-Exp neural networks to approximate the MPC-related cost function given the measurements of the system dynamics. However, in contrast to the proposed DPC method, the implementation of the method in [11] still relies on the online nonlinear optimization, does not handle state constraints, and training on arbitrary reference signals poses computational challenges.

We demonstrate the capabilities of the proposed DPC method on experimental results using a laboratory device called FlexyAir

* Corresponding author.
E-mail addresses: jan.drgona@pnnl.gov (J. Drgoňa), karol.kis@stuba.sk (K. Kiš), aaron.tuor@pnnl.gov (A. Tuor), draguna.vrabie@pnnl.gov (D. Vrabie), martin.klauco@stuba.sk (M. Klaučo).

https://doi.org/10.1016/j.jprocont.2022.06.001
0959-1524/© 2022 Battelle Memorial Institute and The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

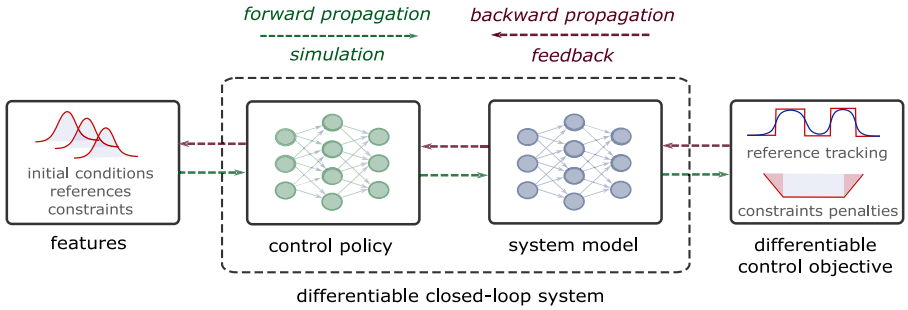


Fig. 1. Conceptual methodology of the proposed nonlinear differentiable predictive control (DPC).

with nonlinear dynamics and noisy measurement signals. We demonstrate several key features of DPC resulting in data-efficiency, scalability, and constraints handling:

1. Data-driven explicit solution of constrained optimal control problems for nonlinear systems by means of differentiable parametric optimization.
2. Differentiable closed-loop system model based on constrained neural control policies and neural state-space models.
3. In contrast with explicit MPC, our method supports dynamical constraints and trajectory preview capabilities.
4. Linear scalability in terms of the number of decision variables and length of the prediction horizon compared to exponential scalability of explicit MPC solutions based on multi-parametric programming.
5. Our approach requires less online computation time and memory than explicit MPC solutions.

1.1. Related work

Explicit model predictive control. Explicit MPC is a powerful method for the systematic design of optimal control policies with a fast online evaluation procedure. In explicit MPC, instead of solving the corresponding optimal control problem online, the solution is pre-computed offline through the parametric optimization [12–15]. The resulting controller then boils down to obtaining a set of piecewise affine (PWA) functions that can be evaluated on an embedded platform. Unfortunately, the explicit MPC comes with a significant drawback: scalability of the solution and the memory footprint of the resulting PWA control policy [16]. Even for small-scale dynamical systems, the explicit MPC's memory footprint can reach several megabytes. In the last years, researchers developed many methods for decreasing the complexity and memory footprint of the explicit solution [17–25]. Nevertheless, all of the aforementioned methods are often costly in terms of processing time and do not scale to large dimensions. Moreover, most of the explicit MPC methods handle only linear system dynamics and constraints. For dealing with nonlinear systems, there are few methods based on approximations of the underlying nonlinear multiparametric programs (mpNLP) [26–28]. While the authors in [29] incorporated ReLU neural networks as constraints in multiparametric programming problems (mpP). Despite these numerous advances, all of the contemporary explicit MPC methods suffer the curse of dimensionality associated with the solution of the underlying mpP [30]. In this paper, we present a scalable alternative by leveraging nonlinear system identification, differentiable programming [31], and sampling of the parametric space to obtain an approximate solution of the underlying mpP.

Approximate MPC. Recently, several works have been devoted to the use of machine learning to approximate the solution of the underlying multiparametric programming problem to obtain the explicit MPC control laws. One set of these works includes supervised learning based on mimicking the MPC behavior and replacing the control law with a neural network [7,32,33]. These approaches are known in the machine learning community as imitation learning [34], or MPC-guided policy search [35]. Several procedures were designed to approximate MPC's behavior with neural networks, which significantly reduces the implementation requirements of the MPC with minimal impact on the control performance [7,33]. Constraint satisfaction using neural network-based approximations of MPC control laws is a new and active research area. To tackle this issue, some authors proposed including additional layers in the policy network projecting the control inputs onto the constrained region of the state and action spaces [36,37]. The authors in [1] use an additional *dual policy* neural network to estimate the sub-optimality of the learned control law with probabilistic guarantees. Others employ learning bounds for empirically validating constraints handling capabilities after the network is trained [5]. For a more comprehensive overview of the approximate MPC and learning-based MPC (LBMPC) methods, we refer the reader to [38]. However, all previous works rely upon several factors, which are prerequisites for their successful implementation. First, the supervisory MPC strategy needs to be designed to generate the training data. This work presents an alternative unsupervised policy optimization approach that does not require a supervisory controller to imitate. Instead, the proposed strategy learns the system dynamics model and uses this model as part of the differentiable closed-loop system model. We will show that this allows for unsupervised sampling of the parametric space generating a closed-loop dataset that can be used in direct policy optimization via stochastic gradient descent.

Data-driven and neural models in MPC. To tackle the modeling problem, some researchers focus on training neural networks as prediction models for MPC [39]. The authors in [40] use low-rank features of the high-dimensional system to train a recurrent neural network (RNN) to predict the control relevant quantities for MPC. Other authors have used structured neural network models inspired by classical linear time-varying state-space models [41], deep Koopman representations [42–44], ensemble of recurrent neural networks (RNN) [45], or RNNs with features selected via principal component analysis (PCA) [46], whereas some have proposed using convex neural architectures [47], graph neural networks [48], or stable neural networks based on Lyapunov functions [49]. In this work, we build on the these trends in nonlinear system identification and employ neural state-space models [50] as generic abstractions for learning nonlinear dynamics models of the controlled system.

Automatic differentiation in control. The use of automatic differentiation (AD) for control and optimization is a well-established method. For instance, CasADi toolbox [51] uses known system dynamics and constraints for constructing a computational graph and computes the gradients for nonlinear optimization solvers. From the perspective of machine learning, the authors in [47] investigated the idea of solving the optimal control problem by backpropagation through the learned system model parametrized via convex neural networks [52]. Others developed, domain-specific differentiable physical models for the robotics domain [53,54]. Learning linear Model Predictive Control (MPC) policies by differentiating the Karush–Kuhn–Tucker (KKT) conditions of the convex approximation at a fixed point was introduced in [55]. However, a generic method for explicit solution of constrained nonlinear optimal control problems with unknown dynamics using AD is still lacking. The presented work aims to bridge this gap by providing unifying algorithmic and open-source software framework [56].

Constrained deep learning. Incorporating constraints into deep learning represents multiple challenges such as non-convexity, convergence, and stability of the learning process [57]. Penalty methods and loss function regularizations are the most straightforward way of imposing constraints on the deep neural network outputs and parameters [58–60]. In those methods, the loss function is augmented with additional terms penalizing the violations of soft constraints via slack variables, which typically works well in practice, often outperforming hard constraint methods [61,62]. The authors in [63] presented a method for imposing hard equality and inequality constraints in the context of learning solutions of optimization problems. For control applications, the authors in [64,65] use barrier methods combined with Lyapunov functions to enforce output constraints, stability, and boundedness of the neural network controller. While the authors in [66] use penalty methods for soft constrained reinforcement learning applied to process control application. An alternative to penalty and barrier methods are neural network architectures imposing hard constraints, such as linear operator constraints [67], or architectures with Hamiltonian [68] and Lagrangian [69] structural priors for enforcing energy conservation laws. In this paper, we leverage both, penalty functions for imposing inequality constraints on state and action variables, and neural architecture design imposing hard equality constraints representing temporal dependencies of the system states, also called single shooting formulation.

2. Current state-of-the-art explicit control

One of the aims of the paper is to compare the proposed DPC strategy with current state-of-the-art solutions in the domain of explicit controllers. Therefore, we present the explicit model predictive control (eMPC) approach, which gives a control law in an explicit form, hence the control action is obtained by a simple function evaluation of a piece-wise affine function. Similarly, the implementation of the DPC strategy, also results in explicit control law, but in this case, in the form of a neural network.

2.1. System dynamics

We assume that the controlled process is an unknown partially observable nonlinear dynamical system in discrete time:

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \tag{1a}$$

$$\mathbf{y}_k = g(\mathbf{x}_k) \tag{1b}$$

where $\mathbf{x}_k \in \mathbb{R}^{n_x}$ is the unknown system state, $\mathbf{y}_k \in \mathbb{R}^{n_y}$ is the observed output, and $\mathbf{u_k} \in \mathbb{R}^{n_u}$ is the control input at time k. Furthermore, We assume that we have access to the data generated by the system in the form of input–output tuples:

$$\Xi = \{(\mathbf{y}_k^i, \mathbf{u}_k^i), (\mathbf{y}_{k+1}^i, \mathbf{u}_{k+1}^i), \dots, (\mathbf{y}_{k+N}^i, \mathbf{u}_{k+N}^i)\}, i \in \mathbb{N}_1^n \tag{2}$$

where n is the number of sampled trajectories with N time steps.

2.2. Model predictive control

To construct the eMPC, we first cast a well-known model predictive controller. We follow standard formulation of the MPC with linear models [70], given by a quadratic optimization problem, specifically formed as

$$\min_{\mathbf{u}_0,\dots,\mathbf{u}_{N-1}} \sum_{k=0}^{N-1} \left(\|\mathbf{y}_k - \mathbf{r}\|_{Q_r}^2 + \|\mathbf{u}_k - \mathbf{u}_{k-1}\|_{Q_{du}}^2 \right) \tag{3a}$$

$$\text{s.t.} \quad \mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k, \tag{3b}$$

$$\mathbf{y}_k = C\mathbf{x}_k + D\mathbf{u}_k, \tag{3c}$$

$$\underline{\mathbf{u}}_k \le \mathbf{u}_k \le \overline{\mathbf{u}}_k, \tag{3d}$$

$$\underline{\mathbf{x}}_k \le \mathbf{x}_k \le \overline{\mathbf{x}}_k, \tag{3e}$$

$$\mathbf{x}_0 = \mathbf{x}(t), \tag{3f}$$

$$\mathbf{u}_{-1} = \mathbf{u}(t - T_s). \tag{3g}$$

Here, the objective function is defined as the finite sum of two terms over a prediction horizon N. Both terms are considered as a weighted norm, i.e. $\|\mathbf{a}\|_Q^2 = \mathbf{a}^\intercal Q \mathbf{a}$. Note, that to enforce feasibility of problem (3), the weighting factors Q_{du} and Q_r must be chosen as positive definite and positive semi-definite matrices, respectively. The constraints (3b)–(3e) are enforced for $k \in 0, 1, \dots, N-1$. Note, that linear system dynamics, given by matrices A, B, C, and D, can be extracted by first principle modeling of the unknown system in (1) or with the well-known System Identification methods [71]. Moreover, to enforce reference tracking, the minimization objective, Eq. (3a), includes a term for reference tracking error, $\|\mathbf{y}_k - \mathbf{r}_k\|_{Q_r}^2$, as well as a control rate penalization term, $\|\mathbf{u}_k - \mathbf{u}_{k-1}\|_{Q_{du}}^2$, which is a standard way to enforce offset free tracking [72]. The optimization problem is initialized with current state measurement $\mathbf{x}(t)$, as in (3f), by the value of the control action from previous sampling instant $\mathbf{u}(t - T_s)$, as described in (3g), and by the reference value $\mathbf{r}$ in (3a). Note, that the notation $\mathbf{u}_{-1}$ is valid for $k = 0$, which is necessary to compute the value of the objective function in the initial prediction step.

The control strategy is implemented in the receding horizon fashion, where we solve (3) towards global optimality yielding an optimal sequence of control actions $\mathbf{U}^\star = [\mathbf{u}_0^{\star\intercal}, \dots, \mathbf{u}_{N-1}^{\star\intercal}]^\intercal$, while only the first action is applied to the system. Representation of such an implementation is visualized on Fig. 2.

We formulate the quadratic optimization problem (3) in MATLAB with the YALMIP toolbox [73]. The problem is then solved numerically with the GUROBI solver. Due to the fast dynamics of the controlled system, and that the numerical solution to the MPC problem takes more time compared to the sampling rate T_s, we consider a parametric solution to the optimization problem (3). The parametric solution allows us to evaluate the control law to obtain the optimal control action within allotted time.

2.3. Explicit model predictive control

Parametric optimization theory allows us to create an analytical map between initial conditions of the optimal control problem (3) and the optimal solution $\mathbf{U}^\star$. Specifically, the vector of parameters, thus the initial conditions, is defined as

$$\hat{\xi} = \begin{bmatrix} \mathbf{x}(\mathbf{t}) \\ \mathbf{r} \\ \mathbf{u}(t - T_s) \end{bmatrix}. \tag{4}$$

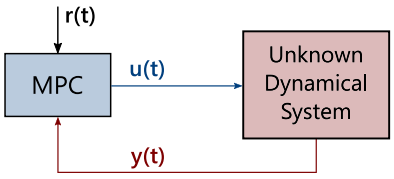


Fig. 2. Realization of the closed-loop control system with an unknown dynamical system.

After applying elementary matrix operations, presented in [74], the OCP from (3) can be reformulated to

$$\min_{\mathbf{U}} \ \mathbf{U}^\top H \mathbf{U} + \hat{\xi}^\top F \mathbf{U} \tag{5a}$$

$$\text{s.t.} \ \ G\mathbf{U} \leq w + S\hat{\xi}, \tag{5b}$$

which constitutes as a parametric quadratic optimization problem (PQP). The solution to this problem can be obtain via standard procedures of parametric programming. Namely, the results is represented by a piece-wise affine (PWA) function, given as

$$\mathbf{U}^\star(\theta) = \begin{cases} \alpha_1 \hat{\xi} + \beta_1 & \text{if } \hat{\xi} \in \mathcal{R}_1 \\ \vdots & \\ \alpha_{n_\mathrm{R}} \hat{\xi} + \beta_{n_\mathrm{R}} & \text{if } \hat{\xi} \in \mathcal{R}_{n_\mathrm{R}}. \end{cases} \tag{6}$$

Here, the variable $\hat{\xi}$ stands for the vector of parameters, n_R denotes the total number of regions, while the vectors α_i and β_i define the specific control law with respect to a region $\mathcal{R}_i$. The regions are defined as polyhedral sets, namely

$$\mathcal{R}_i = \{\hat{\xi} \mid \mathbf{\Gamma}_i \hat{\xi} \leq \mathbf{\Gamma}_i\} \quad i = 1, \ldots, n_\mathrm{R}, \tag{7}$$

Here, the matrices $\mathbf{\Gamma}_i$ and $\mathbf{\Gamma}_i$ denote the half-space representation of regions. Since, the optimization problem (3) is a quadratic problem with linear constraints, all regions are defined by linear inequalities.

Note that the procedure of obtaining a numerical representation of matrices $\mathbf{\Gamma}_i$ and $\mathbf{\Gamma}_i$ is done by means of Multi-Parametric Toolbox [75] in Matlab. To use the explicit MPC in connection with the laboratory device, we export the control law into Python source code. The export to Python source code contains two parts. The first part is the coefficients from (6) and (7), and the second part is the algorithm that evaluates the control law. The algorithm is based on a well-known sequential search method from [76].

3. Differentiable predictive control

This section presents Differentiable Predictive Control (DPC), a neural network-based method for learning constrained explicit predictive control policies from measurement data. The proposed data-driven DPC method has two steps. The first step elaborated in Section 3.1 is a nonlinear system identification based on neural state-space model architecture [50] including constraints to enforce physically realistic predictions. The second step introduced in Section 3.2 is unsupervised offline policy optimization by using the learned neural model of the system dynamics. In particular, we combine the neural state-space model with a neural control policy, constructing a fully parametrized differentiable closed-loop system dynamics model. This generic closed-loop architecture allows us to learn a wide range of constrained control policies using end-to-end auto-differentiation of the MPC-like loss functions and stochastic gradient descent optimization. The high-level overview of the DPC method is provided in Algorithm 1 with conceptual methodology being illustrated in Fig. 1.

Algorithm 1 Differentiable Predictive Control.

input measured system identification dataset Ξ^{ID}
input synthetically sampled control parameters dataset Ξ^{ctrl}
input optimizer $\mathbb{O}$
input Differentiable closed-loop system model composed of neural state space model f_θ and neural control policy π_Θ
input System identification loss ℓ_{ID} with penalty constraints
input MPC-inspired loss ℓ_{DPC} with penalty constraints
train neural state space model f_θ using dataset Ξ^{ID} to optimize loss ℓ_{ID} with optimizer $\mathbb{O}$
train neural policy π_Θ in the closed-loop with the system model f_θ using dataset Ξ^{ctrl} to optimize loss ℓ_{DPC} with optimizer $\mathbb{O}$
return learned system dynamics model f_θ and optimized policy π_Θ

Remark. In the context of explicit MPC, the proposed DPC approach can be seen as unsupervised sampling-based approximate solution of the nonlinear parametric programming problem. Similar to explicit MPC, DPC optimizes control policies offline using N-step ahead predictions of the closed-loop system dynamics model generated as a response to the distribution of synthetically generated control features ξ. After the training, analogous to MPC, DPC is deployed in the receding horizon control (RHC) fashion.

3.1. System identification via constrained neural state space models

We aim to learn a constrained neural representation of the unknown system dynamics, given the input–output time-series dataset (2) obtained from system observation.

Model architecture. We present a generic block neural state-space model (BN-SSM) to represent and learn partially observable unknown nonlinear system dynamics (1), given the labeled dataset (2). The BN-SSM architecture is shown in Fig. 3 with corresponding equations given as follows:

$$\mathbf{x}_{k+1} = f_\mathrm{x}(\mathbf{x}_k) + f_\mathrm{u}(\mathbf{u}_k) \tag{8a}$$

$$\mathbf{y}_k = f_\mathrm{y}(\mathbf{x}_k) \tag{8b}$$

$$\mathbf{x}_0 = f_\mathrm{o}(\mathbf{y}_{1-N}, \ldots, \mathbf{y}_0) \tag{8c}$$

$$k \in \mathbb{N}_0^N \tag{8d}$$

Where k defines the discrete time step, and N defines the prediction horizon, i.e. number of rollout steps of the recurrent model. Individual block components f_x, and f_u, f_y, and f_o are represented by neural networks. Where f_x, and f_u define the model state and input dynamics, replacing the $\mathbf{A}$ and $\mathbf{B}$ matrices in the classical linear state-space model, respectively. The block f_y defines the output mapping from model states $\mathbf{x}_k$ to observables $\mathbf{y}_k$, replacing the $\mathbf{C}$ matrix in the linear model. The observer block f_o maps the past output trajectories $\mathbf{Y}_p = \{\mathbf{y}_{1-N}, \ldots, \mathbf{y}_0\}$ onto initial states $\mathbf{x}_0$, which is necessary for handling partially observable systems. Now let us compactly represent the N-step ahead rollout of the model (8) as $\{\mathbf{Y}_f, \mathbf{X}_f\} = f_\theta^N(\mathbf{Y}_p, \mathbf{U}_f)$ with lumped parameters θ.

Remark. The proposed BN-SSM architecture (8) represents a generalization of a family of neural state-space models [77–84]. Depending on the choice of neural blocks f_x, f_u, f_y, and f_y one can represent fully nonlinear, Hammerstein-Weiner, Weiner, Hammerstein, or simply linear dynamics models with or without internal feedback. Additionally, a block architecture allows us to impose local regularizations on the block structure or constraints on internal block-generated variables.

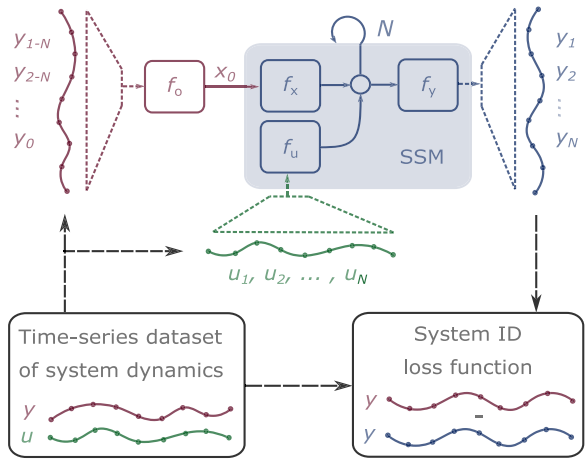


Fig. 3. System identification with block-structured neural state-space model (BN-SSM) (8). Here $\mathbf{y}$ in red and blue color represent observed and predicted system outputs, respectively, $\mathbf{x}$ are model states, and $\mathbf{u}$ are observed control action trajectories.

System identification loss. We train the neural state-space dynamics (8) on sampled input–output trajectories (2) of the observed system dynamics. The multi-term system identification loss is given as follows:

$$\begin{aligned}\mathcal{L}_{\text{ID}}(\mathbf{Y}^{\text{true}}, \mathbf{Y}, \underline{\mathbf{Y}}, \overline{\mathbf{Y}}|\theta) = \\ \frac{1}{nN}\sum_{i=1}^{n}\sum_{k=1}^{N}\Big(\|\mathbf{y}_k^{\text{true},i} - \mathbf{y}_k^i\|_2^2 + Q_{\text{dx}}\|\mathbf{x}_k^i - \mathbf{x}_{k-1}^i\|_2^2 + \\ Q_{\text{y}}\|p(\mathbf{y}_k^i, \underline{\mathbf{y}}_k^i)\|_2^2 + Q_{\text{y}}\|p(\mathbf{y}_k^i, \overline{\mathbf{y}}_k^i)\|_2^2 + \\ Q_{\text{u}}\|p(f_{\text{u}}(\mathbf{u}_k^i), \underline{\mathbf{f}}_{\text{u}})\|_2^2 + Q_{\text{u}}\|p(f_{\text{u}}(\mathbf{u}_k^i), \overline{\mathbf{f}}_{\text{u}})\|_2^2 \Big)\end{aligned} \tag{9}$$

Here k represents time step of the prediction horizon N, and i is the batch index of n sampled trajectories. The first term represents the trajectory tracking loss defined as the two norm over a vector of residuals between the true $\mathbf{Y}^{\text{true}} = \{\mathbf{y}_1^{\text{true},i}, \ldots, \mathbf{y}_N^{\text{true},i}\}$ and predicted $\mathbf{Y} = \{\mathbf{y}_1^i, \ldots, \mathbf{y}_N^i\}$ output trajectories over N steps. The second term is a regularization for smoothing the trajectories by penalizing the one-time step difference between the successive model state $\mathbf{x}$. The third and fourth terms impose constraints on the outputs via penalties. In this work, we employ penalty functions for time-varying lower and upper bounds $\underline{\mathbf{y}}_k, \overline{\mathbf{y}}_k$, respectively, given as follows:

$$\underline{p}(\mathbf{y}_k, \underline{\mathbf{y}}_k) = \max(0,\ -\mathbf{y}_k + \underline{\mathbf{y}}_k) \tag{10a}$$

$$\overline{p}(\mathbf{y}_k, \overline{\mathbf{y}}_k) = \max(0,\ \mathbf{y}_k - \overline{\mathbf{y}}_k) \tag{10b}$$

Furthermore we can leverage the structure of the proposed block neural state-space model (8) and impose similar constraints on the influence of the control input dynamics components $f_{\text{u}}(\mathbf{u}_k)$ as given in the last two terms of the loss function (9).

Remark. Penalty functions in the form (10) can be straightforwardly implemented in modern deep-learning libraries such as PyTorch or Tensorflow using standard ReLU activation functions.

3.2. Differentiable predictive control policy optimization

The objective is to optimize the constrained differentiable predictive control (DPC) policy to govern the unknown dynamical system (1), given the learned neural state-space model (8). We formulate DPC problem as following constrained parametric optimal control problem:

$$\mathcal{L}_{\text{DPC}} = \min_{\Theta} \sum_{k=0}^{N-1} \ell_{\text{DPC}}(\mathbf{y}_k, \mathbf{u}_k, \boldsymbol{\xi}_k) \tag{11a}$$

$$\text{s.t. } \mathbf{x}_{k+1} = f_{\text{x}}(\mathbf{x}_k) + f_{\text{u}}(\mathbf{u}_k),\ \ k \in \mathbb{N}_0^{N-1} \tag{11b}$$

$$\mathbf{y}_k = f_{\text{y}}(\mathbf{x}_k) \tag{11c}$$

$$\mathbf{x}_0 = f_o(\mathbf{y}_{1-N}, \ldots, \mathbf{y}_0) \tag{11d}$$

$$\mathbf{u}_k = \pi_\Theta(\boldsymbol{\xi}_k) \tag{11e}$$

$$\mathbf{x}_k \in \mathcal{X} \tag{11f}$$

$$\mathbf{u}_k \in \mathcal{U} \tag{11g}$$

$$\boldsymbol{\xi}_k \in \Xi^{ctrl} \subset \mathbb{R}^n \tag{11h}$$

Where Θ are decision variables representing weights and biases of the neural control policy $\pi_\Theta(\xi_k)$, and $\ell_{\text{DPC}}(\mathbf{y}_k, \mathbf{u}_k)$ represents MPC-inspired loss function, e.g. reference tracking with constraints violation penalties. In the following paragraphs we elaborate on the design choices of DPC problem (11) implemented and solved as constrained deep learning problem.

Neural control policy. The control parameters ξ sampled from the set Ξ^{ctrl} represent a design choice that captures all decision relevant signals of the control problem. For instance, we can define $\boldsymbol{\xi}_k = \mathbf{x}_k$ as a full state feedback policy. However, in general the neural control policy can take a vector of any relevant control parameters. In this paper we use $\boldsymbol{\xi}_k = \begin{bmatrix}\mathbf{Y}_p^\intercal & \mathbf{R}^\intercal & \underline{\mathbf{Y}}^\intercal & \overline{\mathbf{Y}}^\intercal\end{bmatrix} \in \mathbb{R}^{N\times 3*n_y+n_r}$ with a neural control policy given as:

$$\mathbf{U} = \pi_\Theta(\boldsymbol{\xi}_k) \tag{12}$$

Where $\mathbf{U} = [\mathbf{u}_1, \ldots, \mathbf{u}_N] \in \mathbb{R}^{n_u\times N}$ is an optimal control trajectory, $\mathbf{Y}_p = [\mathbf{y}_{1-N}, \ldots, \mathbf{y}_0] \in \mathbb{R}^{n_y\times N}$ represents observed output trajectories N-steps into the past, $\mathbf{R} = [\mathbf{r}_1, \ldots, \mathbf{r}_N] \in \mathbb{R}^{n_r\times N}$, is a tensor of reference trajectories, while $\underline{\mathbf{Y}} = [\underline{\mathbf{y}}_1, \ldots, \underline{\mathbf{y}}_N] \in \mathbb{R}^{n_y\times N}$, and $\overline{\mathbf{Y}} = [\overline{\mathbf{y}}_1, \ldots, \overline{\mathbf{y}}_N] \in \mathbb{R}^{n_y\times N}$ are imposed lower and upper bounds for future output trajectories, respectively. In this paper, we assume $\boldsymbol{\pi}_\Theta(\boldsymbol{\xi}_k) : \mathbb{R}^m \to \mathbb{R}^n$ to be a fully connected neural network architecture with $l \in \mathbb{N}_1^L$ layers given as:

$$\boldsymbol{\pi}_\Theta(\boldsymbol{\xi}_k) = \mathbf{W}_L\mathbf{h}_L + \mathbf{b}_L \tag{13a}$$

$$\mathbf{h}_l = \boldsymbol{\sigma}(\mathbf{W}_{l-1}\mathbf{h}_{l-1} + \mathbf{b}_{l-1}) \tag{13b}$$

$$\mathbf{h}_0 = \boldsymbol{\xi} \tag{13c}$$

parametrized by $\Theta = \{\mathbf{W}_l, \mathbf{b}_l, |\forall l \in \mathbb{N}_1^L\}$ with weights $\mathbf{W}_l$ and biases $\mathbf{b}_l$, and nonlinear activation function $\boldsymbol{\sigma} : \mathbb{R}^{n_h} \to \mathbb{R}^{n_h}$. In DPC, the neural control policy (12) replaces the PWA control law of explicit MPC described in Section 2.3, thus ξ in DPC the neural policy represents an expanded parametric space compared to the lower-dimensional parametric space of explicit MPC given as Eq. (4).

Differentiable closed-loop system architecture. To train the constrained control policy (12), we design the neural representation of the closed-loop dynamics using the learned neural state-space model f_θ^N (8):

$$\mathbf{U} = \pi_\Theta(\boldsymbol{\xi}_k) \tag{14a}$$

$$\mathbf{Y}_f = f_\theta^N(\mathbf{Y}_p, \mathbf{U}) \tag{14b}$$

The system dynamics model f_θ^N (8) is used to predict N-steps ahead future output trajectories $\mathbf{Y}_f$, given the control actions trajectories $\mathbf{U}$ generated by the neural control policy (12).

The corresponding neural network architecture is shown in Fig. 4. Here the closed-loop model is constructed by connecting

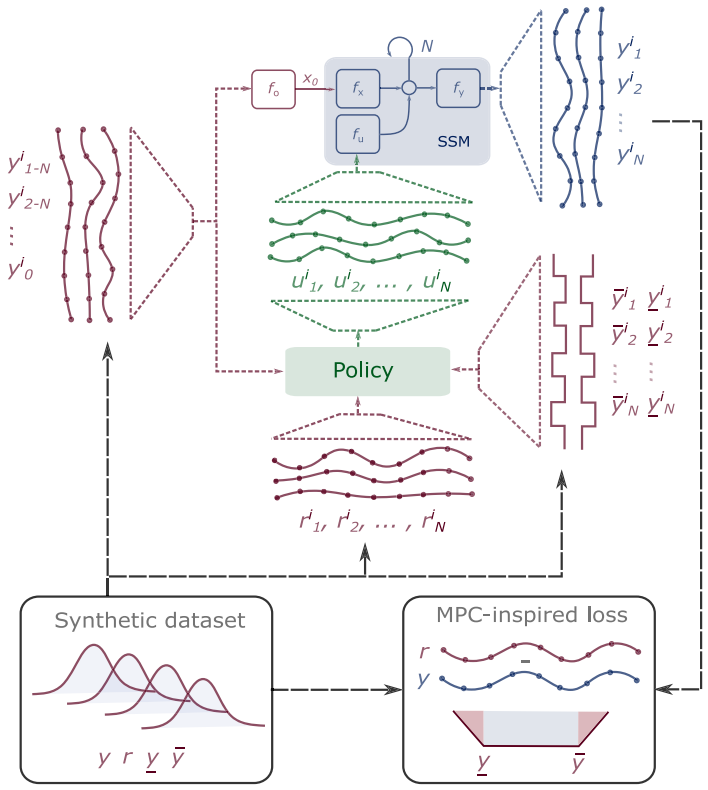


Fig. 4. Differentiable predictive control (DPC) architecture representing problem (11). Here $\mathbf{y}$ represents controlled outputs of the system, $\underline{\mathbf{y}}$ and $\overline{\mathbf{y}}$ represent lower and upper output constraints, $\mathbf{r}$ are sampled reference trajectories, and $\mathbf{u}$ are control actions generated by the neural policy optimized with MPC-inspired loss function.

the learned system dynamics model (8) with the control policy (12), through the control actions trajectories $\mathbf{U}$. Hence the proposed control policy represents a predictive control strategy with a preview of future constraints and reference signals. The policy is optimized by differentiating the closed-loop neural dynamics model on sampled past output trajectories $\mathbf{Y}_p$, and given the forecast of lower and upper constraint trajectories $\underline{\mathbf{Y}}$, and $\overline{\mathbf{Y}}$. The parameters of pre-trained neural state-space model (8) representing the open-loop system dynamics are fixed during the policy optimization. The distribution of past output trajectories $\mathbf{Y}_p$ represents a sampling of initial conditions, while the distribution of time-varying constraints and reference signals represent a sampling of different operational scenarios and tasks.

MPC-inspired loss function. The closed-loop system parametrization (14) now allows us to simulate the effect of varying control features $\boldsymbol{\xi}$ on the system's output dynamics $\mathbf{Y}_f$. This simulation capability, together with differentiability of the closed-loop model (14) is a key feature of the proposed control method which allows us to perform data-driven optimization of neural control policy (12). We train the policy parameters by sampling the distribution of the control features $\boldsymbol{\xi}$, and backpropagating the gradients of the loss function through the closed-loop system model. In particular, we leverage the following MPC-inspired multi-term loss function. The primary reference tracking term is augmented with control smoothing and penalty terms (10) imposed on control actions and output trajectories:

$$\mathcal{L}_{\text{DPC}}(\mathbf{R}, \mathbf{Y}, \underline{\mathbf{Y}}, \overline{\mathbf{Y}}, \underline{\mathbf{U}}, \overline{\mathbf{U}} | \Theta) = \frac{1}{nN} \sum_{i=1}^{n} \sum_{k=1}^{N} \Big(Q_{\text{r}} \|\mathbf{r}_k^i - \mathbf{y}_k^i\|_2^2 + Q_{\text{du}} \|\mathbf{u}_k^i - \mathbf{u}_{k-1}^i\|_2^2 + Q_{\text{y}} \|p(\mathbf{y}_k^i, \underline{\mathbf{y}}_k^i)\|_2^2 + Q_{\text{y}} \|p(\mathbf{y}_k^i, \overline{\mathbf{y}}_k^i)\|_2^2 + Q_{\text{u}} \|p(\mathbf{u}_k^i, \underline{\mathbf{u}}_k^i)\|_2^2 + Q_{\text{u}} \|p(\mathbf{u}_k^i, \overline{\mathbf{u}}_k^i)\|_2^2 \Big) \tag{15}$$

Where k represents time index, N is the prediction horizon, i is the batch index and n is the number of batches of sampled trajectories, respectively. $\mathbf{R}$ represent sampled reference trajectories to be tracked by the output trajectories of the closed-loop system $\mathbf{Y}$, where Q_{r} is reference tracking weight. The second term weighted by Q_{du} represents the control action smoothing. Similar to $\underline{\mathbf{Y}}$, and $\overline{\mathbf{Y}}$, $\underline{\mathbf{U}}$, and $\overline{\mathbf{U}}$ are tensors of lower and upper bounds for the N-step ahead control action trajectories. The penalty terms are weighted by terms Q_{y}, and Q_{u} for output and input constraints, respectively. Now given the loss function (15) we can use the stochastic gradient descent solver to optimize the policy parameters Θ, while keeping the parameters of the dynamics model (8) fixed.

Remark. The proposed control policy optimization procedure does not require an interaction with the real system or its emulator model. Instead, the policy is trained by sampling the closed-loop system using the trained dynamics model. Moreover, the training is extremely data-efficient as all sampled trajectories can be generated synthetically.

4. Experimental case study

This section presents experimental case study of the proposed DPC policy optimization Algorithm 1 implemented in the open-source framework NeuroMANCER [56] for scientific machine learning developed on top of PyTorch [85] deep learning library.

4.1. System description

The presented control approaches are implemented on a laboratory device called FlexyAir.[1] The device is a single-input single-output system, where the actuator is a fan that drives air into a

[1] www.ocl.sk/flexyair.

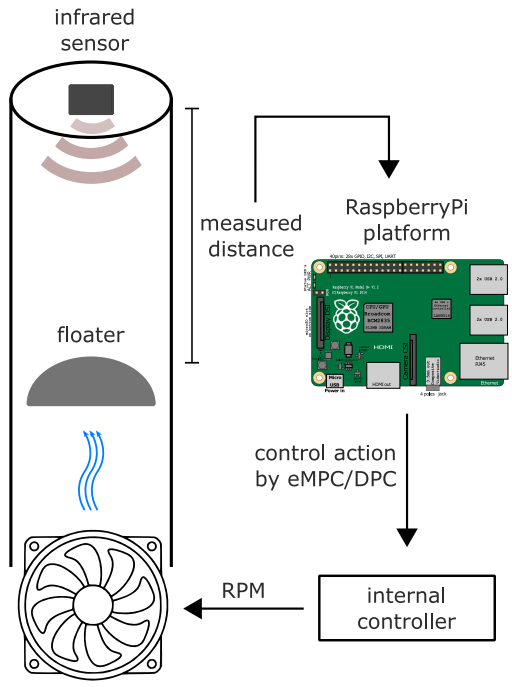


Fig. 5. Sketch of the laboratory device with the Raspberry-Pi platform.

vertical tube with a floater inside. An infrared proximity sensor is placed on the top of the tube, which measures the floater's level. The manipulated variable in this laboratory process is a fan speed command, given to an internal fan speed controller, which sets the corresponding current for the fan itself. Next, the process variable is the position of the floater in the vertical tube. The control objective is to stabilize the floater's vertical distance at the desired reference level, while satisfying given constraints. The sketch of the laboratory device is shown on Fig. 5.

4.2. Dataset

System identification. Our experimental dataset is obtained by observing the real system dynamics with sampling time $T_s = 0.25$ seconds. The measured input–output time series in the form (2) has $m = 9 \cdot 10^3$ datapoints which are used to create training, validation, and test sets with equal lengths of 1000 samples. To take into account time horizons during the training, we apply N-step time shift to generate the past $\mathbf{Y}_p^{\text{true}}$ and future $\mathbf{Y}^{\text{true}}$ tensors for output variables, respectively. The time series in each set are subsequently separated to N-step batches, generating tensors with dimensions $(N, n, n_{\mathbf{y}})$, where n represents number of batches, and $n_{\mathbf{y}}$ is the dimension of the variable $\mathbf{y}$ (the same applies for $\mathbf{u}$). The number of batches $n = \frac{m}{N}$ depends on the total number of datapoints m and length of the prediction horizon N.

Closed-loop control. As mentioned in Section 3.2, the control policy training is based on a sampling of the input sequences of the closed-loop dynamics model and does not require extra measurements of the real system. To demonstrate the data-efficiency, we generate each continuous time series with only $3 \cdot 10^3$ samples for train, validation, and test set, respectively. We apply the same N-step horizon batching as in the case of system identification task. The dataset is also normalized using `min--max` normalization to scale all variables between [0, 1].

The past observations of the output trajectories $\mathbf{Y}_p$ are randomly sampled continuous trajectories, while predicted future trajectories $\mathbf{Y}$ are internally generated by the trained system dynamics model. To improve generalization across dynamic modes, we assume that sampled trajectories $\mathbf{Y}_p$ are dynamically generated sine waves with varying frequency, amplitude, and noise at each optimization epoch. The time-varying references and constraints bounds can be arbitrarily sampled from user-defined distribution to generalize the control across a set of tasks.

In our case, we sample sine waves for the output reference $\mathbf{R}$ with amplitude in range $[0.2, 0.8]$, the lower bound $\underline{\mathbf{Y}}$ in range $[0.1, 0.4]$, and the upper bound $\overline{\mathbf{Y}}$ in range $[0.6, 0.9]$. The control action bounds can be in principle time-varying as well, however, in our case due to the nature of the experimental setup, we assume static constraints $\underline{\mathbf{U}} = 0.0$ and $\overline{\mathbf{U}} = 1.0$

Remark. Training of the control policy on dynamically sampled system output trajectories $\mathbf{Y}_p$ with varying frequencies and amplitudes is inspired by the fact that system response can be decomposed to a set of dynamic modes with fixed frequencies [86]. Alternatively, the trajectories $\mathbf{Y}_p$ could be generated by perturbing the learned system dynamics or simply represented by observations of the real system.

4.3. Metrics

We assess trained DPC performance on two sets of metrics, the first one for training and hyperparameter selection, the second for task-specific performance evaluation. For training, we evaluate the mean squared error (MSE) of system identification loss (9) and policy learning loss (15), respectively. The loss function MSE evaluated on development sets are used for hyperparameter selection, while MSE on test sets are used for performance assessment of the training process. Second, instead of training-oriented processed datasets, we define the task-specific metrics using the real system data with T time steps. For the system identification, we evaluate MSE of the open-loop response of the trained model compared to the response of the real system given as: $\frac{1}{T}\sum_{k=1}^{T} \|\mathbf{y}_k - \mathbf{y}_k^{\text{true}}\|_2^2$. For evaluation of the closed-loop control performance, we compute the reference tracking MSE as $\frac{1}{T}\sum_{k=1}^{T} \|\mathbf{y}_k - \mathbf{r}_k\|_2^2$, and integral of the absolute error (IAE) as $\sum_{k=1}^{T} |\mathbf{y}_k - \mathbf{r}_k|$. For constraints satisfaction, we evaluate mean absolute (MA) value of the output constraints violations: $\frac{1}{T}\sum_{k=1}^{T}\big(|p(\mathbf{y}_k, \underline{\mathbf{y}}_k)| + |p(\mathbf{y}_k, \overline{\mathbf{y}}_k)|\big)$.

4.4. Optimization, and hyperparameters selection

The presented method with structured neural network models was implemented using PyTorch-based [85] toolbox NeuroMANCER [56]. We train our models with randomly initialized weights using Adam optimizer [87] with a learning rate of 0.001. All neural network blocks in our models are designed with `GELU` activation functions [88]. We use a grid search for finding the best performing hyperparameters by assessing the performance of the trained models on the development set and task performance metrics specified in Section 4.3. For the system identification, we modify the search space as presented in [50] to include different neural architectures of the block components (MLP, RNN, ResNet, Linear), and different values for the system ID loss (9) weighting terms (Q_{dx}, Q_y, Q_u). For the neural policy optimization, the search space includes different values for the system DPC loss (15) weighting terms (Q_r, Q_{dx}, Q_y, Q_u). For both the dynamics model and control policy, we select the prediction horizon $N = 32$ steps, which with sampling time $T_s = 0.25$ seconds, corresponds to the 8 seconds time window.

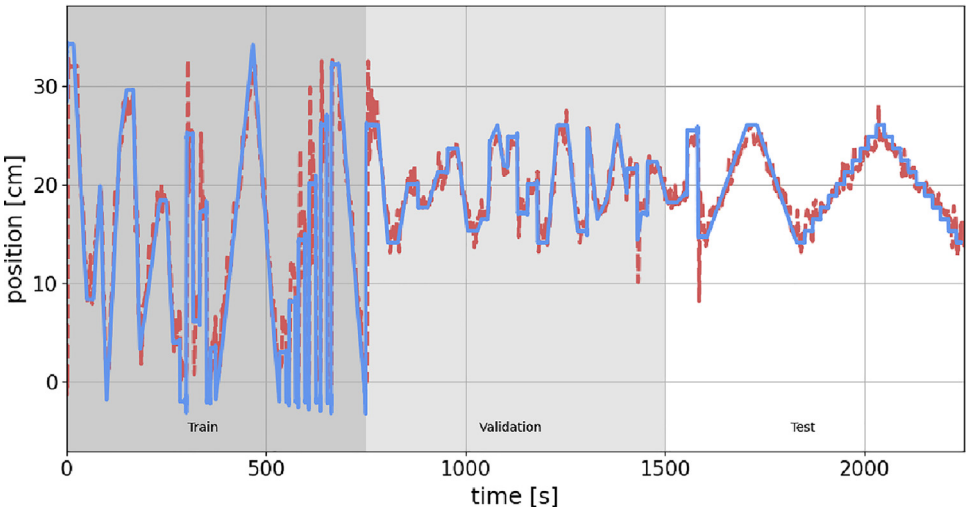


Fig. 6. Open-loop trajectories of the trained neural model (blue) and real system (red).

System identification. The system dynamics model (8) is trained using the system identification dataset on 1000 epochs. The state transition block $f_x : \mathbb{R}^{30} \to \mathbb{R}^{30}$ and the input dynamics block $f_u : \mathbb{R}^1 \to \mathbb{R}^{30}$ are represented by residual neural networks, while output decoder $f_y : \mathbb{R}^{30} \to \mathbb{R}^1$ is a simple linear map. The state encoder map $f_o : \mathbb{R}^1 \to \mathbb{R}^{30}$ is represented by a standard, fully connected neural network. For simplicity, we assume only one step time lag for the state encoder. All individual neural block components are designed with 4 hidden layers and 30 hidden neurons. The resulting neural state-space model has $n_\theta = 24661$ trainable parameters. The weight factors of the system identification loss function (9) are given as follows: $Q_{dx} = 0.2$, $Q_y = 1.0$, $Q_u = 1.0$. The trained block neural state-space model (8) is subsequently used for the design of the constrained differentiable control policy, as described in Section 3.2.

Closed-loop control. The constrained differentiable control policy (12) is trained using synthetically sampled closed-loop system dataset on $5 \cdot 10^3$ epochs with early stopping based on development set MSE.[2] The policy map $\pi_\Theta : \mathbb{R}^{128} \to \mathbb{R}^{32}$ is represented by fully connected neural network with 3 layers each with 20 hidden neurons, resulting in $n_\Theta = 7272$ trainable parameters. The weight factors of the constrained control loss function (15) are $Q_r = 1.0$, $Q_{du} = 0.1$, $Q_y = 2.0$, $Q_u = 10.0$.

In case of the explicit model predictive control (eMPC) strategy (3a), we chose to set the length of the prediction horizon to $N = 5$, while the tuning factors were set to $Q_r = 3$, and $Q_{du} = 4$. Because, construction of classical explicit MPC cannot be done with neural state-space models, the corresponding quadratic optimization problem (3) is prepared using a simplified linear model obtained from the System Identification toolbox in Matlab [89].

Fig. 6 shows open-loop trajectories of the trained neural model compared to the real system. The gray areas divide the dataset to train, development, and test set. Our trained model is capable of closely replicating the output trajectories of a highly noisy nonlinear system. The development set loss MSE is equal to $1.00 \cdot 10^{-3}$ out of which $0.96 \cdot 10^{-3}$ corresponds to the reference tracking loss; the residual is represented by the constraints penalties and regularization. The open-loop reference tracking MSE is equal to $1.35 \cdot 10^{-3}$. Therefore, thanks to the constraints and N-step ahead loss, the model generalizes well beyond the training dataset without accumulating prediction error.

[2] Using early stopping the DPC policy training typically converged using less than 1000 epochs.

4.5. Real-time closed-loop control performance

This section shows the results of the real-time implementation of two control strategies. First, we present a step-change scenario to show the performance concerning transient behavior and reference tracking. Secondly, we introduce an experimental case study, where a harmonic reference alongside a set of harmonic constraints is considered. Here, we show how the proposed differentiable predictive control (DPC) handles constraint satisfaction. All control strategies for these real-time experiments are implemented using an embedded platform running on Raspberry-Pi 3 using Python 3.7.

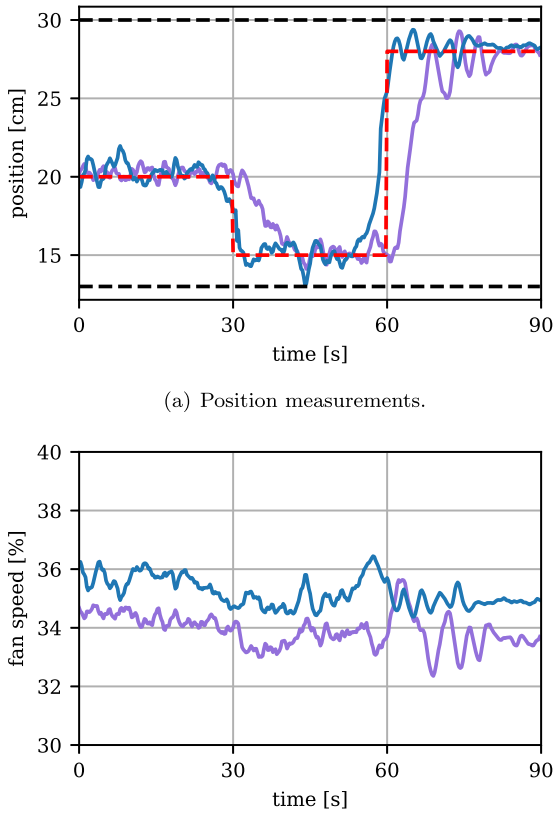
Step-change scenario. This scenario presents the tracking performance when a step change occurs in the process variable. Specifically, we consider a step changes in the reference from $20 \to 15 \to 28$cm for the floater position. Moreover, we set the constraints to $\overline{y} = 30$ cm and bottom $\underline{y} = 13$ cm. Here, we compare the performance of 2 controllers. First, we set the baseline with the explicit MPC strategy. Then we include the proposed DPC policy. The tuning factors of individual controllers are mentioned in Section 4.4.

The control performance of respective policies is visualized on Fig. 7. Furthermore, rigorous validation of measured performances is reported in the Table 1. We can see that both eMPC and DPC follow the trajectory and respect bottom and top constraints from reported results. We can observe that eMPC and DPC react to the reference change in the same fashion (slope of the transient behavior). The difference is just the time instant when the change occurs. This is determined by the length of the prediction horizon of individual policies. While for the DPC the length of the horizon is not a bottleneck (32 samples in this case), the eMPC cannot be constructed for longer horizons than 10 in this case we use $N = 5$ due to the memory footprint. Even though we achieve the same reference value with both tested controllers, we observe a slight offset on the side of manipulated variables (cf. Fig. 7), which can be neglected. Such behavior is caused by integrating dynamics of the laboratory process, i.e., one steady-state value for the manipulated variable can lead to several steady-state values for the controlled variable. Specifically, we can observe that the mean value of the fan speed between times $t \in [30, 60]$ s, which corresponds to the reference of 15 cm, is close to the fan speed value for the reference of 28 cm.

Finally, we direct the reader to the Table 1, where a quantitative numerical evaluation is reported. Here we followed the

Table 1
Qualitative evaluation of control performance, based on metrics given in 4.3.

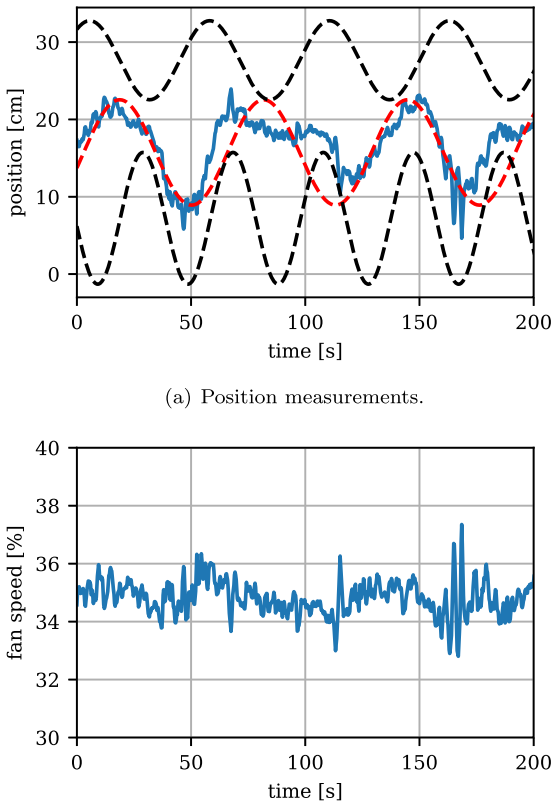
	Reference tracking		Constraints violation	
	MSE	IAE	MSE	MAE
DPC	1.76	265	0	0
eMPC	8.97	526	0	0



(a) Position measurements.

(b) Profile of the manipulated variable.

Fig. 7. Real-time measurement profiles of DPC and eMPC control strategies with static constraints. Top figure shows the reference (red-dashed), explicit model predictive control (purple), constrained differentiable predictive control (blue) and constraints (black-dashed). Bottom figure shows the fan speeds of respective controllers.



(a) Position measurements.

(b) Profile of the manipulated variable.

Fig. 8. Real-time measurement profile of proposed DPC strategy under the influence of harmonic reference and constraints. Top figure shows the reference signal (red-dashed) and measured floater position (blue). Bottom figure shows the profile of the fan speed.

metrics established in Section 4.3. In terms of MSE criterion, the DPC performs almost 8 times better than the explicit MPC. We also include the IAE criterion, which is more common in the control community, and in this case, the DPC outperforms the eMPC by more than 50%. In terms of constraints violations, these criteria read to 0 since neither DPC and eMPC cross the limits.

Constraints satisfaction. The second scenario involves the demonstration of systematic constraints handling by the DPC strategy. Here we utilize a harmonic reference and harmonic constraints. Concrete results are presented in Fig. 8. Note, that the constraints are not violated even if the reference crosses out of allowed space, thus empirically demonstrating remarkable robustness capabilities even in this challenging scenario with high measurement noise.

4.6. Idealized simulation case studies

The purpose of this section is to demonstrate control capabilities of the proposed constrained differentiable predictive control policies (DPC) in idealized simulations. We use the trained system dynamics model for representing the controlled system, omitting the influence of the plant-model mismatch and additive real-time disturbances affecting the dynamics of the real system. We show that, in case of perfect system dynamics model, DPC can achieve offset-free reference tracking capabilities, and robust constraints satisfaction, hence empirically demonstrating convergence to near optimal control performance.

Reference tracking. Fig. 9 shows the offset-free reference tracking capabilities of the trained DPC method assessed using four different dynamic signals. Besides tracking an arbitrary dynamic reference, DPC policy demonstrates predictive control capabilities using a reference preview. In particular, notice the change in the trajectories several time steps before previewed step changes in the reference signals. To be able to react safely and in advance of forecasted parameters such as references and constraints is a desired feature for many industrial control applications. The preview capability also represents additional value compared to explicit MPC, which cannot handle preview of its parameters.

Constraints satisfaction. We demonstrate the capability of DPC to balance conflicting objectives in terms of reference tracking and constraints handling. Fig. 10 plots DPC performance with dynamic reference crossing dynamic constraints. Even in this challenging scenario, the trained DPC policy satisfies the constraints while compromising on the tracking performance of the unattainable reference.

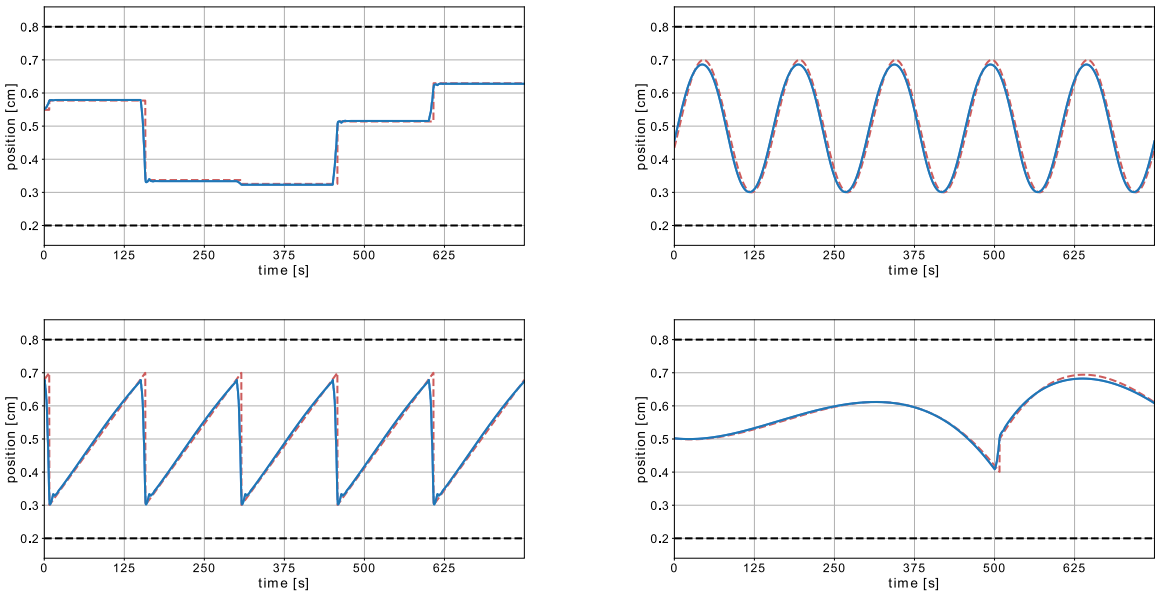


Fig. 9. Simulated closed-loop control trajectories demonstrating reference tracking capabilities of DPC.

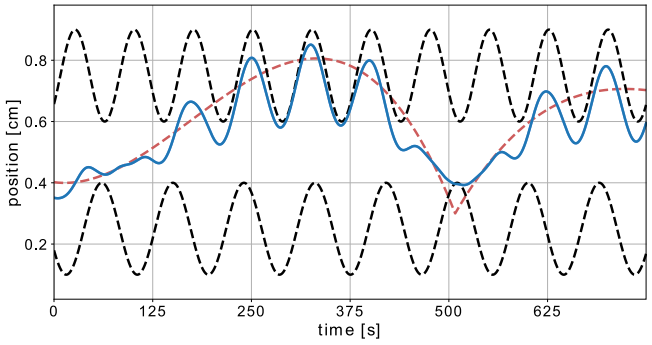


Fig. 10. Simulated closed-loop control trajectories demonstrating balancing conflicting objectives with dynamic reference and dynamic constraints.

4.7. Scalability analysis

In Table 2 we compare the scalability of the proposed differentiable predictive control (DPC) against explicit model predictive control (eMPC) in terms of on-line computational requirements, memory footprint, policy complexity, and off-line construction time with the increasing length of the prediction horizon N.

We compare the mean and maximum online computational (CPU) time and memory footprint required to evaluate and store the DPC and eMPC control policies. As shown in Table 2, the online evaluation of both control policies is extremely fast in terms of CPU time. This property is crucial for controlling fast and highly dynamical systems with high-frequency sampling rates, such as UAVs or agile robotic systems. However, memory requirements are significantly different for linearly scalable DPC compared to exponentially growing requirements of eMPC. In the case of eMPC, its enormous memory demands limit the applicability of this control strategy only to small scale systems with very low prediction horizons. Contrary to eMPC, DPC policies have an extremely low memory footprint regardless of the prediction horizon's length, opening the doors to large-scale practical control applications.

To deeper understand the reported memory requirements, we evaluate the complexity of the control policies in terms of the number of parameters for DPC and the number of critical regions for eMPC.

Table 2 shows that the number of parameters of the DPC scale linearly with the increasing prediction horizon N, while a number of critical regions of eMPC scales exponentially. The reason behind this is that complexity of eMPC policy (6) is primarily given by the number of constraints that scale exponentially with the length of the prediction horizon and the number of optimized variables. On the other hand, the complexity of DPC policy depends mainly on the number of hidden nodes and the number of layers, which allows it to scale to large state–action spaces with long prediction horizons.

Table 2 also reports the construction time for the eMPC policy using multiparametric programming [75,90], to give the reader notions about the limitations of this solution method. Specifically, we direct the attention to the construction time of the eMPC with $N = 10$ for which the generation of the associated controller took almost 3−days. Contrarily, the construction time of DPC scales linearly to a large prediction horizon, as reported in Table 2. Here we demonstrate a tremendous potential of the proposed DPC policies to scale up to large-scale control problems way beyond the reach of classical eMPC.

5. Limitations and future work

Despite experimentally demonstrated robust performance, the proposed DPC method as described in this paper does not provide theoretical guarantees on the constraints satisfaction or closed-loop system stability. This is due to the use of soft constraints via penalty methods. Future work includes using online safety filters for learning-based controllers [91], gradient projection methods [63], or the use of control barrier functions [92] to guarantee constraints satisfaction. From a theoretical perspective, structural similarity of the underlying optimization problem to well studied MPC problem presents opportunities for further extensions of the proposed DPC methodology. Some examples include adaptive control via online updates of the nonlinear system model, robust and stochastic control via constraints tightening, or closed-loop stability guarantees based on stability of MPC [93–95]. Additional extensions include offset-free formulation with augmented state space model and control error integrator dynamics recently proposed for neural network-based controllers [96]. From the

Table 2
Scalability analysis of the proposed differentiable predictive control (DPC) policy against explicit MPC (eMPC).

N	5	7	10	12	15
	Mean CPU time [ms]				
DPC	0.369	0.355	0.371	0.380	0.502
eMPC	0.455	0.472	0.429	–	–
	Max CPU time [ms]				
DPC	6.978	7.978	7.945	8.066	5.026
eMPC	1.325	1.927	4.684	–	–
	Memory footprint [kB]				
DPC	13	15	17	19	21
eMPC	611	9300	65200	–	–
	Number of policy parameters				
DPC	1845	2247	2850	3252	3855
	Number of policy regions				
eMPC	108	347	1420	2631	5333
	Construction time [h]				
DPC[a,b]	0.1	0.1	0.1	0.2	0.2
eMPC[c]	0.1	4.0	66.5	–	–

[a]Trained on 1000 epochs without early stopping.
[b]Computed on Core i7 2.6 GHz CPU with 16GB RAM.
[c]Computed on Core i7 4.0 GHz CPU with 32GB RAM.

system identification perspective, the presented work is limited to discrete-time time-invariant nonlinear dynamical systems. Various neural architectures might be more or less suitable to parametrize the proposed neural state space models, depending on the underlying dynamical system. Straightforward model architecture extensions of the proposed methodology may include a whole range of differentiable models, for instance, sparse identification of nonlinear dynamics (SINDy) [97], multistep neural networks [98], or graph neural networks [99].

6. Conclusions

We have experimentally demonstrated that it is possible to train constrained optimal control policies purely based on the observations of the dynamics of the unknown nonlinear system. The principle is based on optimizing control policies with constraints penalty functions by differentiating trained neural state-space models representing an internal model of the observed nonlinear system. We denote this control approach constrained differentiable predictive control (DPC).

We compare the performance of trained DPC policy against classical explicit model predictive control (eMPC). The control algorithms are implemented on a laboratory device with the Raspberry-Pi platform. In comparison with eMPC using a linear model, DPC achieves better control performance due to the nonlinear system dynamics model and the reference and constraints preview capability. However, most importantly, DPC has unprecedented scalability beyond the limitations of eMPC. DPC scales well with increased problem complexity defined by the length of the prediction horizon resulting in an increased number of decision variables and constraints of the underlying optimization problem. DPC demonstrates linear scalability in terms of memory footprint, number of policy parameters, and required construction time. Therefore, we believe that the proposed DPC method has the potential for wide adoption in large-scale control systems with limited computational resources and fast sampling rates.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the Mathematics for Artificial Reasoning in Science (MARS) initiative via the Laboratory Directed Research and Development (LDRD) investments at Pacific Northwest National Laboratory (PNNL). PNNL is a multiprogram national laboratory operated for the U.S. Department of Energy (DOE) by Battelle Memorial Institute under Contract No. DE-AC05-76RL0-1830.

K. Kiš and M. Klaučo gratefully acknowledge the contribution of the Slovak Research and Development Agency under the project APVV-20-0261 and by the Scientific Grant Agency of the Slovak Republic under the grant VEGA 1/0545/20.

References

[1] X. Zhang, M. Bujarbaruah, F. Borrelli, Near-optimal rapid MPC using neural networks: A primal-dual policy learning framework, IEEE Trans. Control Syst. Technol. (2020) 1–13.
[2] S. Lucia, B. Karg, A deep learning-based approach to robust nonlinear model predictive control, IFAC-PapersOnLine 51 (20) (2018) 511–516, 6th IFAC Conference on Nonlinear Model Predictive Control NMPC 2018, URL http://www.sciencedirect.com/science/article/pii/S2405896318326958.
[3] S. Chen, K. Saulnier, N. Atanasov, D.D. Lee, V. Kumar, G.J. Pappas, M. Morari, Approximating explicit model predictive control using constrained neural networks, in: 2018 Annual American Control Conference, ACC, 2018, pp. 1520–1527.
[4] E.T. Maddalena, C.G. da S. Moraes, G. Waltrich, C.N. Jones, A neural network architecture to learn explicit MPC controllers from data, 2019, http://arxiv.org/abs/1911.10789 [arXiv:1911.10789].
[5] M. Hertneck, J. Köhler, S. Trimpe, F. Allgöwer, Learning an approximate model predictive controller with guarantees, IEEE Control Syst. Lett. 2 (3) (2018) 543–548.
[6] S. Lucia, D. Navarro, B. Karg, H. Sarnago, O. Lucía, Deep learning-based model predictive control for resonant power converters, IEEE Trans. Ind. Inf. 17 (1) (2021) 409–420.
[7] Y. Lohr, M. Klaučo, M. Fikar, M. Mönnigmann, Machine learning assisted solutions of mixed integer MPC on embedded platforms, in: Preprints of the 21st IFAC World Congress, Vol. 21, (Virtual), Berlin, Germany, July 12-17, 2020, 2020, URL https://www.uiam.sk/assets/publication_info.php?id_pub=2192.
[8] J. Drgoňa, D. Picard, M. Kvasnica, L. Helsen, Approximate model predictive building control via machine learning, Appl. Energy 218 (2018) 199–216, URL http://www.sciencedirect.com/science/article/pii/S0306261918302903.
[9] U. Rosolia, X. Zhang, F. Borrelli, Data-driven predictive control for autonomous systems, Annu. Rev. Control Robot. Autonom. Syst. 1 (1) (2018) 259–286, URL https://doi.org/10.1146/annurev-control-060117-105215.
[10] J. Drgona, A. Tuor, D. Vrabie, Learning constrained adaptive differentiable predictive control policies with guarantees, 2020, arXiv, URL https://arxiv.org/abs/2004.11184.

[11] S. Brüggemann, C. Possieri, On the use of difference of log-sum-exp neural networks to solve data-driven model predictive control tracking problems, IEEE Control Syst. Lett. 5 (4) (2021) 1267–1272.
[12] E.N. Pistikopoulos, V. Dua, N.A. Bozinis, A. Bemporad, M. Morari, On-line optimization via off-line parametric optimization tools, Comput. Chem. Eng. 26 (2) (2002) 175–185, URL http://www.sciencedirect.com/science/article/pii/S0098135401007396.
[13] A. Bemporad, M. Morari, V. Dua, E.N. Pistikopoulos, The explicit linear quadratic regulator for constrained systems, Automatica 38 (1) (2002) 3–20, URL https://www.sciencedirect.com/science/article/pii/S0005109801001741.
[14] D. Tavernini, M. Metzler, P. Gruber, A. Sorniotti, Explicit nonlinear model predictive control for electric vehicle traction control, IEEE Trans. Control Syst. Technol. 27 (4) (2019) 1438–1451.
[15] R. Oberdieck, N.A. Diangelakis, E.N. Pistikopoulos, Explicit model predictive control: A connected-graph approach, Automatica 76 (2017) 103–112, URL https://www.sciencedirect.com/science/article/pii/S0005109816303971.
[16] M. Kvasnica, J. Hledík, I. Rauová, M. Fikar, Complexity reduction of explicit model predictive control via separation, Automatica 49 (6) (2013) 1776–1781, URL https://www.uiam.sk/assets/publication_info.php?id_pub=1378.
[17] F. Scibilia, S. Olaru, M. Hovd, Approximate explicit linear MPC via Delaunay tessellation, in: 2009 European Control Conference, ECC, 2009, pp. 2833–2838.
[18] M. Kvasnica, J. Löfberg, M. Fikar, Stabilizing polynomial approximation of explicit MPC, Automatica 47 (10) (2011) 2292–2297, URL https://www.uiam.sk/assets/publication_info.php?id_pub=1171.
[19] M. Kvasnica, M. Fikar, Clipping-based complexity reduction in explicit MPC, IEEE Trans. Automat. Control 57 (7) (2012) 1878–1883.
[20] M. Kvasnica, B. Takács, J. Holaza, S. Di Cairano, On region-free explicit model predictive control, in: 54rd IEEE Conference on Decision and Control, 54, Osaka, Japan, 2015, pp. 3669–3674.
[21] J. Drgoňa, M. Klaučo, F. Janeček, M. Kvasnica, Optimal control of a laboratory binary distillation column via regionless explicit MPC, Comput. Chem. Eng. (2017) 139–148.
[22] M. Kvasnica, P. Bakaráč, M. Klaučo, Complexity reduction in explicit MPC: A reachability approach, Systems Control Lett. 124 (2019) 19–26, URL https://www.uiam.sk/assets/publication_info.php?id_pub=1980.
[23] S. Hovland, J.T. Gravdahl, Complexity reduction in explicit MPC through model reduction, IFAC Proc. Vol. 41 (2) (2008) 7711–7716, 17th IFAC World Congress, URL https://www.sciencedirect.com/science/article/pii/S1474667016401874.
[24] N.A. Nguyen, M. Gulan, S. Olaru, P. Rodriguez-Ayerbe, Convex lifting: Theory and control applications, IEEE Trans. Automat. Control 63 (5) (2018) 1243–1258.
[25] C.N. Jones, M. Morari, Polytopic approximation of explicit model predictive controllers, IEEE Trans. Automat. Control 55 (11) (2010) 2542–2553.
[26] I. Pappas, N.A. Diangelakis, E.N. Pistikopoulos, Multiparametric/explicit nonlinear model predictive control for quadratically constrained problems, J. Process Control 103 (2021) 55–66, URL https://www.sciencedirect.com/science/article/pii/S0959152421000706.
[27] T. Johansen, On multi-parametric nonlinear programming and explicit nonlinear model predictive control, in: Proceedings of the 41st IEEE Conference on Decision and Control, Vol. 3 2002, 2002, pp. 2768–2773 vol.3.
[28] P. Petsagkourakis, C. Theodoropoulos, Data driven reduced order nonlinear multiparametric MPC for large scale systems, in: A. Friedl, J.J. Klemeš, S. Radl, P.S. Varbanov, T. Wallek (Eds.), 28th European Symposium on Computer Aided Process Engineering, in: Computer Aided Chemical Engineering, vol. 43, Elsevier, 2018, pp. 1249–1254, URL https://www.sciencedirect.com/science/article/pii/B9780444642356502175.
[29] J. Katz, I. Pappas, S. Avraamidou, E.N. Pistikopoulos, Integrating deep learning models and multiparametric programming, Comput. Chem. Eng. 136 (2020) 106801, URL https://www.sciencedirect.com/science/article/pii/S0098135419311378.
[30] I. Pappas, D. Kenefake, B. Burnak, S. Avraamidou, H. Ganesh, J. Katz, N.A. Diangelakis, E. Pistikopoulos, Multiparametric programming in process systems engineering: Recent developments and path forward, in: Frontiers in Chemical Engineering, 2020.
[31] M. Innes, A. Edelman, K. Fischer, C. Rackauckas, E. Saba, V.B. Shah, W. Tebbutt, A differentiable programming system to bridge machine learning and scientific computing, 2019, CoRR abs/1907.07587, arXiv:1907.07587.
[32] B. Karg, S. Lucia, Approximate moving horizon estimation and robust nonlinear model predictive control via deep learning, Comput. Chem. Eng. 148 (2021) 107266, URL https://www.sciencedirect.com/science/article/pii/S0098135421000442.
[33] K. Kiš, M. Klaučo, Neural network based explicit MPC for chemical reactor control, Acta Chim. Slovaca 12 (2) (2019) 218–223, URL https://www.uiam.sk/assets/publication_info.php?id_pub=2115.
[34] I. Mordatch, E. Todorov, Combining the benefits of function approximation and trajectory optimization, in: Robotics: Science and Systems, RSS, 2014.
[35] T. Zhang, G. Kahn, S. Levine, P. Abbeel, Learning deep control policies for autonomous aerial vehicles with MPC-guided policy search, 2015, CoRR abs/1509.06791, arXiv:1509.06791.
[36] S. Chen, K. Saulnier, N. Atanasov, D.D. Lee, V. Kumar, G.J. Pappas, M. Morari, Approximating explicit model predictive control using constrained neural networks, in: 2018 Annual American Control Conference, ACC, 2018, pp. 1520–1527.
[37] P.L. Donti, M. Roderick, M. Fazlyab, J.Z. Kolter, Enforcing robust control guarantees within neural network policies, in: The Ninth International Conference on Learning Representations, ICLR, 2021.
[38] L. Hewing, J. Kabzan, M.N. Zeilinger, Cautious model predictive control using Gaussian process regression, IEEE Trans. Control Syst. Technol. 28 (6) (2020) 2736–2743.
[39] I. Lenz, R.A. Knepper, A. Saxena, DeepMPC: Learning deep latent features for model predictive control, in: Robotics: Science and Systems, 2015.
[40] K. Bieker, S. Peitz, S.L. Brunton, J.N. Kutz, M. Dellnitz, Deep model predictive control with online learning for complex physical systems, 2019, CoRR abs/1905.10094, arXiv:1905.10094.
[41] A. Broad, I. Abraham, T.D. Murphey, B.D. Argall, Structured neural network dynamics for model-based control, 2018, CoRR abs/1808.01184, arXiv:1808.01184.
[42] B. Lusch, J.N. Kutz, S.L. Brunton, Deep learning for universal linear embeddings of nonlinear dynamics, 9 (1), p. 4950, http://dx.doi.org/10.1038/s41467-018-07210-0.
[43] M. Korda, I. Mezić, Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control, Automatica 93 (2018) 149–160, URL https://www.sciencedirect.com/science/article/pii/S000510981830133X.
[44] S. Peitz, S.E. Otto, C.W. Rowley, Data-driven model predictive control using interpolated koopman generators, SIAM J. Appl. Dyn. Syst. 19 (3) (2020) 2162–2193, http://dx.doi.org/10.1137/20M1325678.
[45] Z. Wu, A. Tran, D. Rincon, P.D. Christofides, Machine learning-based predictive control of nonlinear processes. Part I: Theory, AIChE J. 65 (11) (2019) e16729, URL https://aiche.onlinelibrary.wiley.com/doi/abs/10.1002/aic.16729.
[46] H. Hassanpour, B. Corbett, P. Mhaskar, Integrating dynamic neural network models with principal component analysis for adaptive model predictive control, Chem. Eng. Res. Des. 161 (2020) 26–37, URL https://www.sciencedirect.com/science/article/pii/S0263876220301337.
[47] Y. Chen, Y. Shi, B. Zhang, Optimal control via neural networks: A convex approach, 2018.
[48] Y. Li, J. Wu, J.-Y. Zhu, J.B. Tenenbaum, A. Torralba, R. Tedrake, Propagation networks for model-based control under partial observation, in: ICRA, 2019.
[49] Y.-C. Chang, N. Roohi, S. Gao, Neural Lyapunov control, in: H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché Buc, E. Fox, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 32, Curran Associates, Inc., 2019, pp. 3245–3254, URL http://papers.nips.cc/paper/8587-neural-lyapunov-control.pdf.
[50] E. Skomski, S. Vasisht, C. Wight, A. Tuor, J. Drgoňa, D. Vrabie, Constrained block nonlinear neural dynamical models, in: 2021 American Control Conference, ACC, 2021, pp. 3993–4000.
[51] J.A.E. Andersson, J. Gillis, G. Horn, J.B. Rawlings, M. Diehl, CasADi: A software framework for nonlinear optimization and optimal control, Math. Program. Comput. 11 (1) (2019) 1–36, http://dx.doi.org/10.1007/s12532-018-0139-4.
[52] B. Amos, L. Xu, J.Z. Kolter, Input convex neural networks, 2016, CoRR abs/1609.07152, arXiv:1609.07152.
[53] F. de Avila Belbute-Peres, K. Smith, K. Allen, J. Tenenbaum, J.Z. Kolter, End-to-end differentiable physics for learning and control, in: S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 31, Curran Associates, Inc., 2018, pp. 7178–7189.
[54] J. Degrave, M. Hermans, J. Dambre, F. Wyffels, A differentiable physics engine for deep learning in robotics, 2016, CoRR abs/1611.01652, arXiv:1611.01652.
[55] B. Amos, I.D.J. Rodriguez, J. Sacks, B. Boots, J.Z. Kolter, Differentiable MPC for end-to-end planning and control, 2018, CoRR abs/1810.13400, arXiv:1810.13400.
[56] A. Tuor, J. Drgona, E. Skomski, NeuroMANCER: Neural modules with adaptive nonlinear constraints and efficient regularizations, 2021, URL https://github.com/pnnl/neuromancer.
[57] T. Yang, Advancing non-convex and constrained learning: Challenges and opportunities, AI Matt. 5 (3) (2019) 29–39, http://dx.doi.org/10.1145/3362077.3362085.
[58] D. Pathak, P. Krähenbühl, T. Darrell, Constrained convolutional neural networks for weakly supervised segmentation, 2015, CoRR abs/1506.03648, arXiv:1506.03648.
[59] Z. Jia, X. Huang, E.I. Chang, Y. Xu, Constrained deep weak supervision for histopathology image segmentation, IEEE Trans. Med. Imaging 36 (11) (2017) 2376–2388.

[60] C.K. Goh, Y. Liu, A.W.K. Kong, A constrained deep neural network for ordinal regression, in: 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2018, pp. 831–839.
[61] P. Márquez-Neila, M. Salzmann, P. Fua, Imposing hard constraints on deep networks: Promises and limitations, 2017, CoRR abs/1706.02025, arXiv:1706.02025.
[62] H. Kervadec, J. Dolz, J. Yuan, C. Desrosiers, E. Granger, I.B. Ayed, Log-barrier constrained CNNs, 2019, CoRR abs/1904.04205, arXiv:1904.04205.
[63] P. Donti, D. Rolnick, J.Z. Kolter, DC3: A learning method for optimization with hard constraints, in: International Conference on Learning Representations, 2021.
[64] Y. Liu, C. Su, H. Li, R. Lu, Barrier function-based adaptive control for uncertain strict-feedback systems within predefined neural network approximation sets, IEEE Trans. Neural Netw. Learn. Syst. 31 (8) (2020) 2942–2954.
[65] K. Zhao, J. Chen, Adaptive neural quantized control of MIMO nonlinear systems under actuation faults and time-varying output constraints, IEEE Trans. Neural Netw. Learn. Syst. 31 (9) (2020) 3471–3481.
[66] O. Dogru, N. Wieczorek, K. Velswamy, F. Ibrahim, B. Huang, Online reinforcement learning for a continuous space system with experimental validation, J. Process Control 104 (2021) 86–100, URL https://www.sciencedirect.com/science/article/pii/S0959152421000950.
[67] J. Hendriks, C. Jidling, A. Wills, T. Schön, Linearly constrained neural networks, IEEE Trans. Neural Netw. Learn. Syst. (2020) Submitted for publication.
[68] S. Greydanus, M. Dzamba, J. Yosinski, Hamiltonian neural networks, 2019, CoRR abs/1906.01563, arXiv:1906.01563.
[69] M. Lutter, C. Ritter, J. Peters, Deep Lagrangian networks: Using physics as model prior for deep learning, 2019, CoRR vol. abs/1907.04490, arXiv:1907.04490.
[70] D.Q. Mayne, J.B. Rawlings, C.V. Rao, P.O.M. Scokaert, Constrained model predictive control: Stability and optimality, Automatica 36 (6) (2000) 789–814, URL http://www.sciencedirect.com/science/article/pii/S0005109899002149.
[71] L. Ljung, System Identification, Theory for the User, second ed., Prentice Hall PTR, Sweden, 1999.
[72] G. Pannocchia, Robust disturbance modeling for model predictive control with application to multivariable ill-conditioned processes, J. Process Control 13 (8) (2003) 693–701, URL http://www.sciencedirect.com/science/article/pii/S0959152402001348.
[73] J. Löfberg, YALMIP : A Toolbox for modeling and optimization in MATLAB, in: Proc. of the CACSD Conference, Taipei, Taiwan, 2004, Available from http://users.isy.liu.se/johanl/yalmip/.
[74] F. Borrelli, A. Bemporad, M. Morari, Predictive Control for Linear and Hybrid Systems, Cambridge University Press, 2017, URL https://books.google.de/books?id=cdQoDwAAQBAJ.
[75] M. Herceg, M. Kvasnica, C. Jones, M. Morari, Multi-parametric toolbox 3.0, in: 2013 European Control Conference, Zurich, Switzerland, 2013, pp. 502–510.
[76] B. Takács, J. Števek, R. Valo, M. Kvasnica, Python code generation for explicit MPC in MPT, in: European Control Conference 2016, Aalborg, Denmark, 2016, pp. 1328–1333, URL https://www.uiam.sk/assets/publication_info.php?id_pub=1737.
[77] R.G. Krishnan, U. Shalit, D. Sontag, Structured inference networks for nonlinear state space models, in: AAAI'17: Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, 2017.
[78] D. Hafner, T.P. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, J. Davidson, Learning latent dynamics for planning from pixels, 2018, CoRR abs/1811.04551, arXiv:1811.04551.
[79] O.P. Ogunmolu, X. Gu, S.B. Jiang, N.R. Gans, Nonlinear systems identification using deep dynamic neural networks, 2016, CoRR abs/1610.01439, arXiv:1610.01439.
[80] S.S. Rangapuram, M.W. Seeger, J. Gasthaus, L. Stella, Y. Wang, T. Januschowski, Deep state space models for time series forecasting, in: S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, R. Garnett (Eds.), Advances in Neural Information Processing Systems, Vol. 31, Curran Associates, Inc., 2018, pp. 7785–7794.
[81] J.-S. Wang, Y.-C. Chen, A Hammerstein-Wiener recurrent neural network with universal approximation capability, in: 2008 IEEE International Conference on Systems, Man and Cybernetics, 2008, pp. 1832–1837.
[82] D. Masti, A. Bemporad, Learning nonlinear state-space models using deep autoencoders, in: 2018 IEEE Conference on Decision and Control, CDC, 2018, pp. 3862–3867.
[83] A. Tuor, J. Drgona, D. Vrabie, Constrained neural ordinary differential equations with stability guarantees, 2020.
[84] J. Schoukens, L. Ljung, Nonlinear system identification: A user-oriented roadmap, 2019, CoRR abs/1902.00683, arXiv:1902.00683.
[85] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., Pytorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, 2019, pp. 8024–8035.
[86] P. Schmid, Dynamic mode decomposition of numerical and experimental data, J. Fluid Mech. 656 (2008) 5–28.
[87] D.P. Kingma, J. Ba, Adam: A method for stochastic optimization, 2014, arXiv preprint arXiv:1412.6980.
[88] D. Hendrycks, K. Gimpel, Bridging nonlinearities and stochastic regularizers with Gaussian error linear units, 2016, CoRR abs/1606.08415, arXiv:1606.08415.
[89] L. Ljung, System Identification: Theory for the User, second ed., Prentice-Hall, Upper Saddle River, NJ, 1999, p. 607.
[90] R. Oberdieck, N. Diangelakis, I. Nascu, M. Papathanasiou, M. Sun, S. Avraamidou, E. Pistikopoulos, On multi-parametric programming and its applications in process systems engineering, Chem. Eng. Res. Des. (2016).
[91] K.P. Wabersich, M.N. Zeilinger, A predictive safety filter for learning-based control of constrained nonlinear dynamical systems, Automatica 129 (2021) 109597, URL https://www.sciencedirect.com/science/article/pii/S0005109821001175.
[92] W. Shaw Cortez, D. Oetomo, C. Manzie, P. Choong, Control barrier functions for mechanical systems: Theory and application to robotic grasping, IEEE Trans. Control Syst. Technol. 29 (2) (2021) 530–545.
[93] D. Mayne, J. Rawlings, C. Rao, P. Scokaert, Constrained model predictive control: Stability and optimality, Automatica 36 (6) (2000) 789–814, URL https://www.sciencedirect.com/science/article/pii/S0005109899002149.
[94] A. Zheng, M. Morari, Stability of model predictive control with mixed constraints, IEEE Trans. Automat. Control 40 (10) (1995) 1818–1823.
[95] A. Bemporad, M. Morari, Robust model predictive control: A survey, in: A. Garulli, A. Tesi (Eds.), Robustness in Identification and Control, Springer London, London, 1999, pp. 207–226.
[96] P. Pauli, J. Köhler, J. Berberich, A. Koch, F. Allgöwer, Offset-free setpoint tracking using neural network controllers, in: Proceedings of the 3rd Conference on Learning for Dynamics and Control, in: Proceedings of Machine Learning Research, vol. 144, PMLR, 2021, pp. 992–1003, URL http://proceedings.mlr.press/v144/pauli21a.html.
[97] S.L. Brunton, J.L. Proctor, J.N. Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems, Proc. Natl. Acad. Sci. 113 (15) (2016) 3932–3937, URL https://www.pnas.org/content/113/15/3932.
[98] M. Raissi, P. Perdikaris, G.E. Karniadakis, Multistep neural networks for data-driven discovery of nonlinear dynamical systems, 2018, arXiv preprint arXiv:1801.01236.
[99] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, P.W. Battaglia, Learning to simulate complex physics with graph networks, in: ICML, 2020.